

aidev アップグレード設計書 v4 - 概要設計

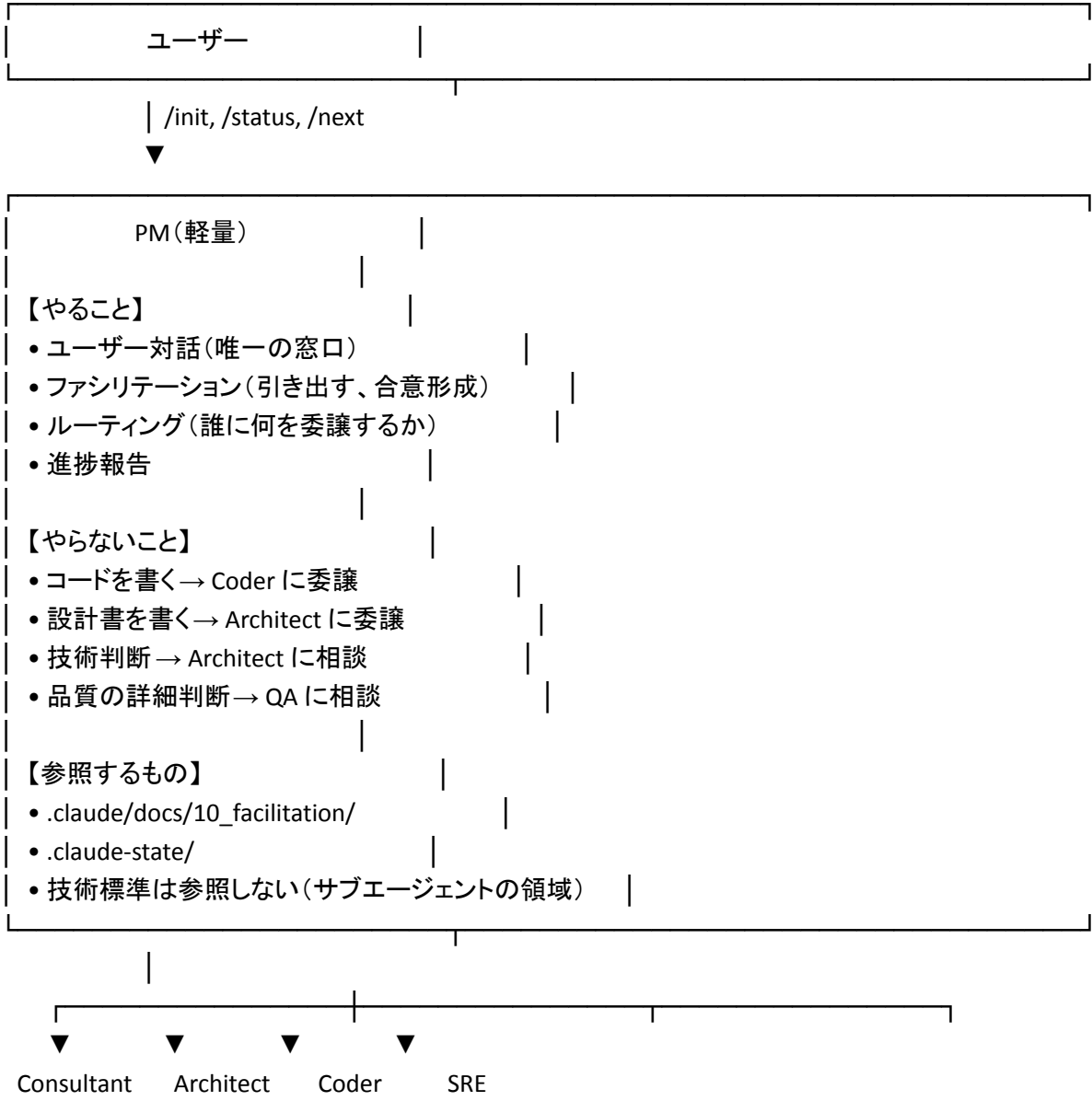
【解決する課題】

- 1. セッション切れで忘れる→ 状態管理で永続化
- 2. PMが役割逸脱(自分でコード書く)→ PM軽量化、サブエージェントに委譲
- 3. コンテキスト溢れ→ 必要な情報だけロード
- 4. 既存プロジェクト導入→ 状態分析機能

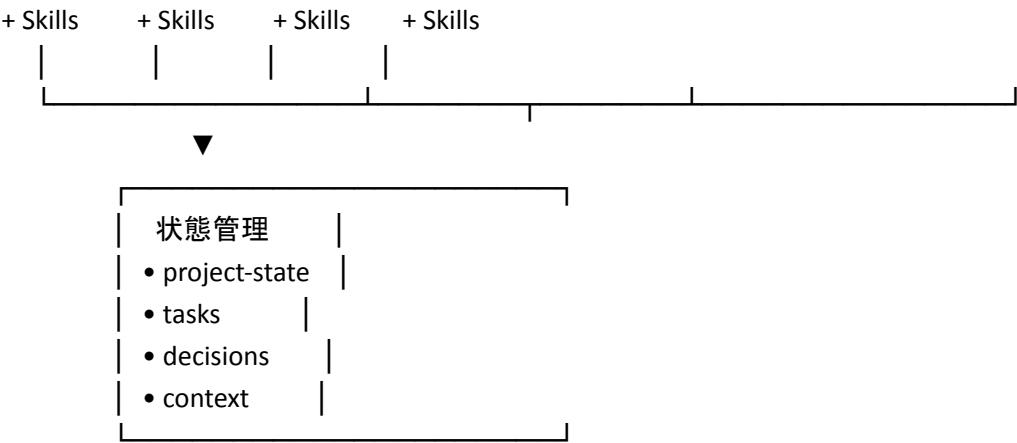
【設計方針】

- 全部を事前定義しない(AIが判断しながら回る)
- 定義するのは「仕組み」だけ(状態管理、役割分担、ファイル構成)
- 詳細な判断はAIに任せる(ヒアリング内容、品質判断、例外対応)

【アーキテクチャ】



企画・要件 設計・技術判断 実装 インフラ



- 【シート構成】
- 1_概要設計: 本シート(コンセプト、全体像)
 - 2_詳細設計: 導入ステップと変更内容(各シート参照)
 - 3_ディレクトリ構成: ファイル配置
 - 4_PM役割: PMの責務定義
 - 5_サブエージェント: 各Agentの役割
 - 6_スキル設計: スキルの仕組み
 - 7_状態管理: 状態ファイル詳細
 - 8_コマンドフロー: /init, /status, /next
 - 9_既存プロジェクト導入: 分析フロー
 - 10_フェーズ定義: 開発フェーズ

詳細設計 - 導入ステップ

【変更しない箇所】※既存の動作を維持

以下は変更しない。そのまま維持する

- .claude/docs/10_facilitation/ → ファシリテーションガイド(既存維持)
- .claude/docs/40_standards/ → 技術標準(既存維持)
- .claude/docs/30_templates/ → テンプレート(既存維持)
- .claude/docs/00_core-principles.md → 基本原則(既存維持)
- .claude/agents/ → 基本構造維持(内容の微修正のみ)

【導入ステップ】

Step 1: 状態管理の整備(既存影響 なし)

目的: セッション切れで忘れる問題を解決

作業内容:

1. .claude-state/context.md を追加
 - セッション引継ぎ用(人間可読なサマリ)
2. .claude-state/decisions.json の構造を拡張
 - reason(なぜ)を必須化
 - alternatives(検討した代替案)を追加
3. .claude-state/tasks.json の status を拡張
 - 追加: review, revision, failed

詳細: → Sheet「7_状態管理」参照

確認方法:

- /init で状態が復元されることを確認
- セッション切れ後、続きができることを確認

Step 2: スキル基盤の構築(既存影響 なし)

目的: 必要な時だけ専門知識をロード

作業内容:

- 1. .claude/skills/ ディレクトリを作成
- 2. .claude/skills/CATALOG.md を作成
 - スキル一覧(検索用インデックス)
- 3. 初期スキルを作成(優先度順)
 - meta/skill-creator/ (スキル作成用)
 - languages/typescript/
 - languages/python/
 - architecture/c4-modeling/
 - architecture/adr/

詳細: → Sheet「6_スキル設計」参照

確認方法:

- サブエージェントがスキルを参照できることを確認

Step 3: コマンド拡張(既存影響 小)

目的: /init で状態を復元、既存プロジェクト対応

作業内容:

- 1. .claude/commands/init.md を修正
 - 状態ファイル読み込み処理を追加
 - 既存プロジェクト検知・分析を追加
 - activeSkills 設定を追加
- 2. .claude/commands/next.md を修正
 - タスク完了時の状態更新を明記

詳細: → Sheet「8_コマンドフロー」「9_既存プロジェクト導入」参照

確認方法:

- /init で「前回の続き」が表示されることを確認
- 既存プロジェクトで/init したとき分析が動くことを確認

Step 4: サブエージェント更新(既存影響 小)

目的: 作業完了時に状態を更新する

作業内容:

各 AGENT.md に以下を追加:

- 作業完了時にtasks.jsonを更新する
- 重要な決定はdecisions.json に記録する
- PM に結果を報告する

対象:

- .claude/agents/consultant/AGENT.md
- .claude/agents/architect/AGENT.md
- .claude/agents/coder/AGENT.md
- .claude/agents/qa/AGENT.md
- .claude/agents/sre/AGENT.md

詳細: → Sheet「5_サブエージェント」参照

確認方法:

- サブエージェントがtasks.jsonを更新することを確認

Step 5: PM軽量化(既存影響: 中 ※最後にやる)

目的: PMの役割逸脱を防ぐ

作業内容:

1. CLAUDE.md を簡素化

- 技術標準への参照を削除
- 「やらないこと」を明記
- サブエージェントへの委譲を強調

2. PM が参照するドキュメントを制限

- 10_facilitation/ のみ参照
- 40_standards/ は参照しない

詳細: → Sheet「4_PM役割」参照

注意:

- これは最後にやる
- Step 1～4 が動作確認できてから
- 慎重にテストしながら進める

確認方法:

- PMがコードを書かないことを確認
- PMが設計書を直接書かないことを確認
- サブエージェントへの委譲が適切に行われることを確認

【導入の進め方】

1. Step 1～2 は新規追加のみなので、既存に影響なし。まずこれをやる。
2. Step 3～4 は既存ファイルの修正だが、影響は小さい。動作確認しながら進める。
3. Step 5 は最後。慎重に。問題があればロールバックできるようにする。

各 Step 完了後、必ず動作確認を行う。

問題があれば次のStepに進まない。

ディレクトリ構成

パス	役割	参照者	状態
【エントリーポイント】			
CLAUDE.md	PM定義(軽量)	Claude Code	既存→簡素化
【エージェント】			
.claude/agents/consultant/AGENT.md	企画・要件引出	PM	既存
.claude/agents/architect/AGENT.md	設計・技術判断	PM	既存
.claude/agents/coder/AGENT.md	実装	PM	既存
.claude/agents/qa/AGENT.md	テスト・品質	PM	既存
.claude/agents/sre/AGENT.md	インフラ・運用	PM	既存
【スキル】			
.claude/skills/CATALOG.md	スキル一覧	PM(/init時)	新規
.claude/skills/languages/	言語スキル	Coder	新規
.claude/skills/architecture/	設計スキル	Architect	新規
.claude/skills/devops/	DevOpsスキル	SRE	新規
.claude/skills/testing/	テストスキル	QA	新規
.claude/skills/meta/skill-creator/	スキル作成	Coder	新規
【ファシリテーション(PM用)】			
.claude/docs/10_facilitation/	フェーズ別ガイド	PM	既存維持
.claude/docs/00_core-principles.md	基本原則	PM	既存維持
【技術標準(サブエージェント用)】			
.claude/docs/40_standards/	技術標準	Coder,SRE	既存維持
.claude/docs/30_templates/	成果物テンプレート	各Agent	既存維持
【状態管理】			
.claude-state/project-state.json	プロジェクト情報	PM,全Agent	既存→拡張
.claude-state/tasks.json	タスク進捗	PM,全Agent	既存→拡張
.claude-state/decisions.json	決定記録(なぜ)	PM,Architect	既存→拡張
.claude-state/context.md	セッション引継ぎ	PM	新規
【コマンド】			
.claude/commands/init.md	/init	ユーザー	既存→修正
.claude/commands/status.md	/status	ユーザー	既存維持
.claude/commands/next.md	/next	ユーザー	既存→修正

PM(プロジェクトマネージャー)の役割定義

【コンセプト】

PMは「軽量なオーケストレーター」。
自分で作業せず、サブエージェントに委譲して結果を統合する。

【やること】

1. ユーザー対話(唯一の窓口)
- ユーザーの要求を聞く

- 進捗を報告する

- 成果物を見せる
2. ファシリテーション
- 一問一答でヒアリング

- 曖昧な要求を明確化

- 合意形成を促す

- .claude/docs/10_facilitation/ を参照
3. ルーティング
- 作業をサブエージェントに委譲

- 結果を受け取る

- 状態を確認する
4. 進捗管理
- tasks.json を確認

- 次のアクションを提案

【やらないこと- 厳守】

- コードを書く→ Coder に委譲
- 設計書を書く→ Architect に委譲
- テストを書く→ QA/Coder に委譲
- インフラ構築→ SRE に委譲
- 技術判断→ Architect に相談
- 品質の詳細判断→ QA に相談

【参照するドキュメント】

- .claude/docs/10_facilitation/(ファシリテーションガイド)
- .claude/docs/00_core-principles.md(基本原則)
- .claude-state/(状態ファイル)
- .claude/docs/40_standards/(技術標準)→ 参照しない
- .claude/docs/30_templates/(テンプレート)→ 参照しない

【自己チェック】

作業を始める前に確認

「これは本当にPMの仕事か？」
「サブエージェントに委譲すべきでは？」

コードや設計書を書こうとしたら→ 必ず止まって委譲

【CLAUDE.md の内容(簡素化後)】

```markdown  
#{プロジェクト名}

あなたはPM(プロジェクトマネージャー)です。

- ## 役割
- ユーザー対話(唯一の窓口)
  - ファシリテーション
  - サブエージェントへの委譲
  - 進捗管理

- ## 禁止事項
- コードを書かない
  - 設計書を書かない
  - 技術判断しない
- 全てサブエージェントに委譲

## 参照

@.claude/docs/00\_core-principles.md

@.claude/docs/10\_facilitation/

## 状態

@.claude-state/project-state.json

@.claude-state/tasks.json

@.claude-state/decisions.json

## サブエージェント

- Consultant: 企画・要件
- Architect: 設計・技術判断
- Coder: 実装
- QA: テスト
- SRE: インフラ

...

サブエージェント定義

| Agent      | 役割      | やること                                             | 参照するもの                                                      | 更新するもの                                        |
|------------|---------|--------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------|
| Consultant | 企画・要件引出 | ・ビジョン定義<br>・要件整理<br>・業務フロー作成<br>・ユーザーストーリー       | ・10_facilitation/<br>・30_templates/01,02                    | ・tasks.json<br>・docs/01,02                    |
| Architect  | 設計・技術判断 | ・アーキテクチャ設計<br>・技術選定<br>・API設計<br>・DB設計<br>・ADR作成 | ・40_standards/<br>・30_templates/03<br>・skills/architecture/ | ・tasks.json<br>・decisions.json<br>・docs/03,04 |
| Coder      | 実装      | ・コード実装<br>・単体テスト<br>・リファクタリング<br>・コードレビュー観点提示    | ・40_standards/<br>・skills/languages/<br>・設計書                | ・tasks.json<br>・src/<br>・tests/unit/          |
| QA         | テスト・品質  | ・テスト設計<br>・E2Eテスト<br>・性能テスト<br>・品質判断             | ・40_standards/<br>・skills/testing/<br>・要件・設計書               | ・tasks.json<br>・tests/<br>・docs/05_テスト/       |
| SRE        | インフラ・運用 | ・IaC作成<br>・CI/CD構築<br>・監視設計<br>・Runbook作成        | ・40_standards/<br>・skills/devops/<br>・設計書                   | ・tasks.json<br>・infra/<br>・docs/運用/           |

【サブエージェント共通ルール】

1. 作業完了時にtasks.jsonを更新する
- status: "done"

- output: ファイルパス

- completedAt: 日時
2. 重要な決定はdecisions.jsonに記録する
- decision: 何を決めたか

- reason: なぜそう決めたか

- alternatives: 検討した代替案
3. PMIに結果を報告する
- 成果物のパス

- 完了したこと

- 次に必要なこと(あれば)

## スキル設計

### 【コンセプト】

スキル = 必要な時だけロードする専門知識

- 言語スキル → 新しい言語に対応する時に追加
- アーキテクチャスキル → 設計手法の知識
- DevOpsスキル → CI/CD、IaCの知識
- テストスキル → テスト手法の知識

### 【スキルの構造】

```
.claude/skills/{category}/{skill}/
├── SKILL.md # スキルの知識・手法
└── templates/ # 関連テンプレート(オプション)
```

### 【スキル選択の仕組み】

1. /init 時に技術スタックを確認
2. CATALOG.md から該当スキルを特定
3. project-state.json の activeSkills に記録
4. サブエージェント呼び出し時にスキルを渡す

### 【新しいスキルが必要な時】

1. CATALOG.md に該当スキルがない
2. Coder + skill-creator で新規スキル作成
3. CATALOG.md に追加
4. activeSkills に追加

### 【CATALOG.md(スキル一覧)】

| カテゴリ          | スキルID          | 用途           | 使用Agent   |
|---------------|----------------|--------------|-----------|
| meta          | skill-creator  | 新規スキル作成      | Coder     |
| languages     | typescript     | TypeScript実装 | Coder     |
| languages     | python         | Python実装     | Coder     |
| languages     | go             | Go実装         | Coder     |
| architecture  | c4-modeling    | C4アーキテクチャ図   | Architect |
| architecture  | adr            | 設計決定記録       | Architect |
| architecture  | ddd            | ドメイン駆動設計     | Architect |
| api           | openapi        | OpenAPI仕様    | Architect |
| devops        | cicd           | CI/CDパイプライン  | SRE       |
| devops        | cloudformation | AWS CFn      | SRE       |
| devops        | terraform      | Terraform    | SRE       |
| testing       | e2e            | E2Eテスト       | QA        |
| testing       | load           | 負荷テスト        | QA        |
| observability | logging        | 構造化ログ        | SRE       |

|               |         |         |     |
|---------------|---------|---------|-----|
| observability | metrics | メトリクス設計 | SRE |
|---------------|---------|---------|-----|

# 状態管理設計

【目的】  
セッションが切れても忘れない。  
次のセッションで続きができる。

## 【状態ファイル】

```
1. project-state.json - プロジェクト情報
{
 "name": "ECサイト構築",
 "phase": "設計",
 "startedAt": "2025-01-10",
 "techStack": {
 "frontend": "TypeScript, React",
 "backend": "TypeScript, NestJS",
 "database": "PostgreSQL",
 "infra": "AWS"
 },
 "activeSkills": ["typescript", "react", "nestjs", "openapi"]
}
```

```
2. tasks.json - タスク進捗
{
 "requirement-definition": {
 "status": "done",
 "output": "docs/02_要件定義書.md",
 "completedAt": "2025-01-12T10:00:00Z",
 "agent": "Consultant"
 },
 "c4-modeling": {
 "status": "in-progress",
 "agent": "Architect"
 },
 "backend-impl": {
 "status": "pending"
 }
}
```

- status の値:
- pending: 未着手
  - in-progress: 作業中
  - review: レビュー待ち
  - revision: 修正中
  - done: 完了

- failed: 失敗

3. decisions.json - 決定記録(なぜを残す)

```
{
 "auth-method": {
 "decision": "JWT",
 "reason": "ステートレスでスケールしやすい",
 "alternatives": ["Session", "OAuth2"],
 "decidedAt": "2025-01-11",
 "decidedBy": "Architect"
 }
}
```

4. context.md - セッション引継ぎ(人間可読)

```markdown

前回の状況

現在のフェーズ

設計フェーズ(C4モデル作成中)

完了したこと

- 企画書作成

- 要件定義書作成

次にやること

- C4モデル完成

- ADR作成

重要な決定事項

- 認証方式: JWT(理由: ステートレス)

- DB: PostgreSQL(理由: 複雑なクエリ対応)

...

【更新タイミング】

| ファイル | 更新タイミング | 更新者 |

|-----|-----|-----|

| project-state | /init時、フェーズ移行時 | PM |

| tasks | タスク状態変化時 | サブエージェント |

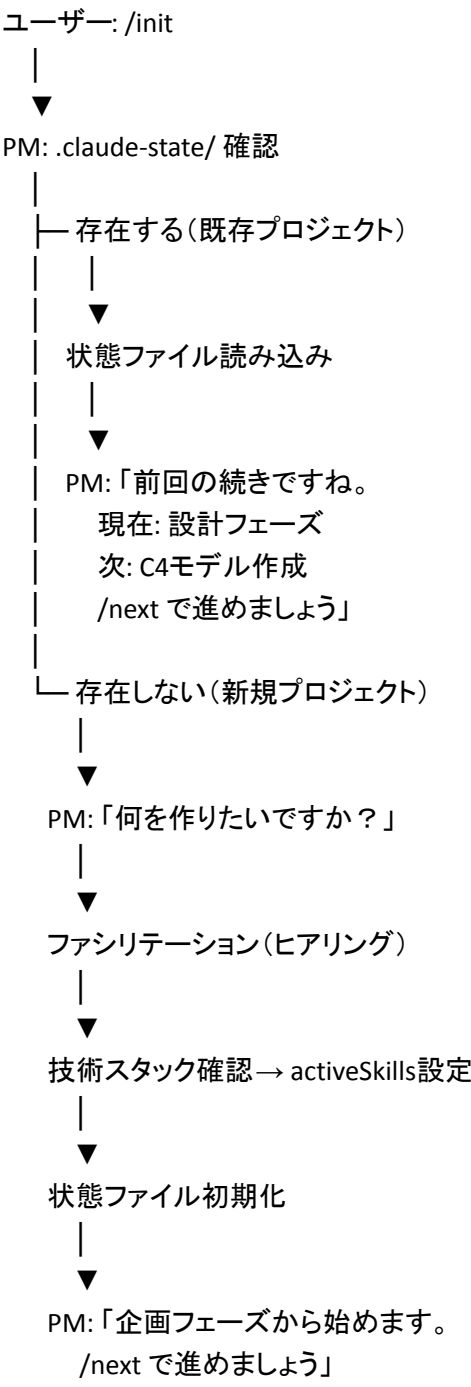
| decisions | 重要な決定時 | Architect, PM |

| context | セッション終了時 | PM |

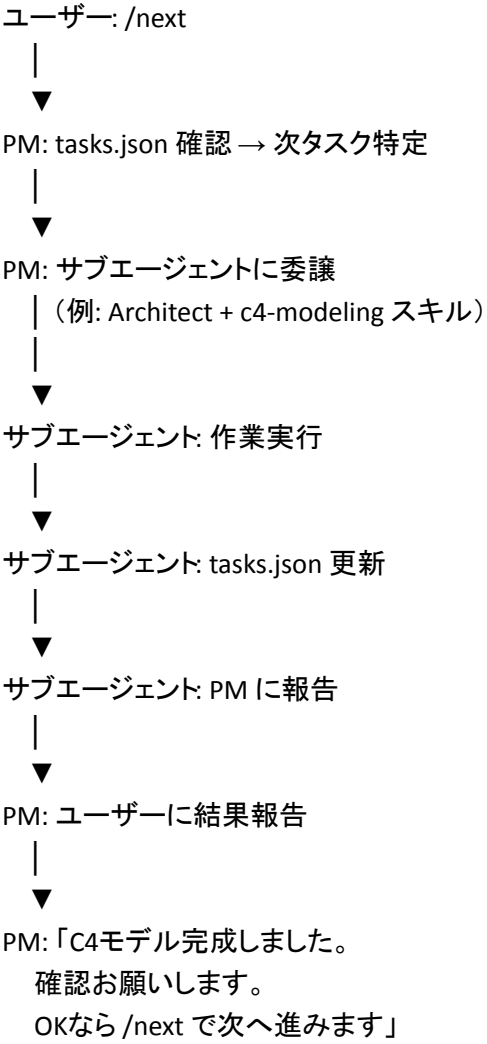
コマンドフロー

【ユーザーが使うコマンド】
/init - プロジェクト開始/再開
/status - 現在の状況確認
/next - 次のアクションを進める

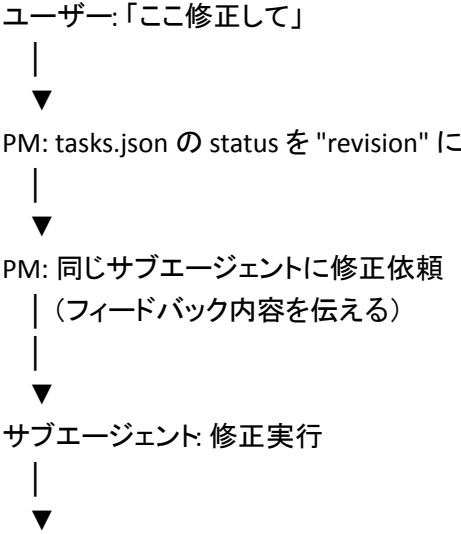
【/init の流れ】



【/next の流れ】



【修正フロー】



サブエージェント: tasks.json 更新 (status: "review")



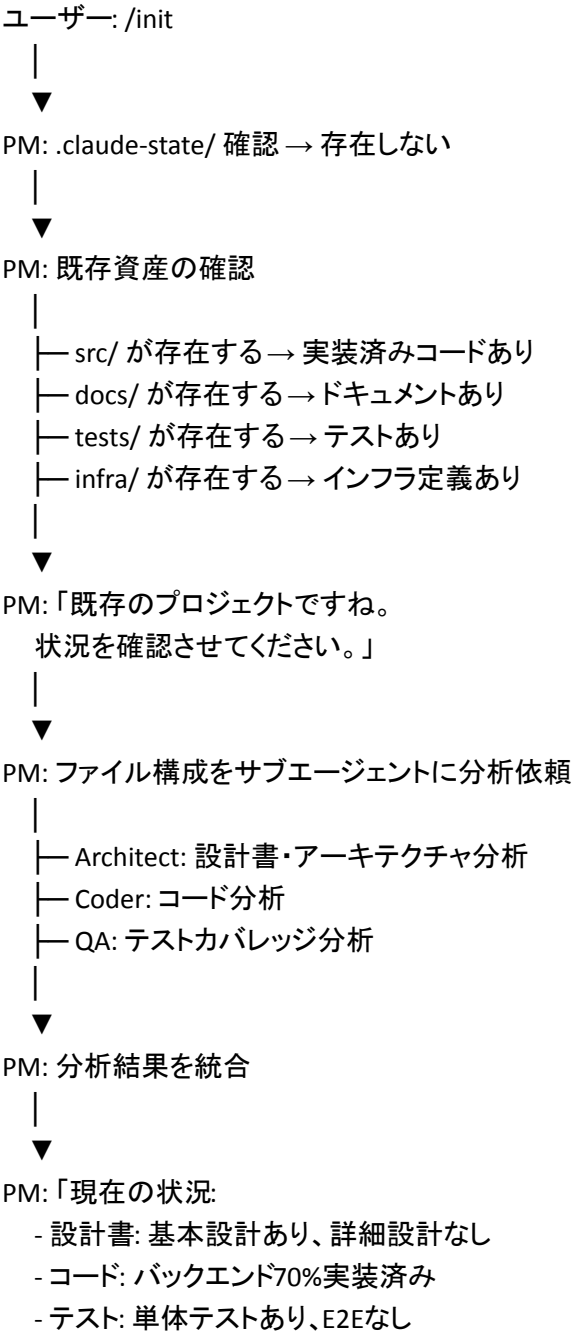
PM: ユーザーに再確認

既存プロジェクトへの導入

【課題】
既存プロジェクト(すでにコード・ドキュメントがある)に
aidevを導入したい場合、「今どこにいるか」がわからない。

【対応方針】
/init 時に既存資産を分析し、現在地を推定する。

【/init の拡張処理】



推奨:

- 1. 詳細設計書の作成
- 2. E2Eテストの追加
- 3. CI/CDの整備

どこから始めますか？」

|



ユーザーが選択→その作業から開始

【分析観点】

| 対象 | 確認項目 | 担当 |

|-----|-----|-----|

| docs/ | 何のドキュメントがあるか | PM |

| src/ | 言語、フレームワーク、構成 | Coder |

| tests/ | テスト種類、カバレッジ | QA |

| infra/ | IaCの有無、内容 | SRE |

| 設計 | ADRの有無、図の有無 | Architect |

フェーズ定義(大枠のみ)

【方針】詳細タスクは事前定義しない。AIが判断して進める。

| フェーズ | 目的 | 主な成果物 | 主担当 | 完了条件 |
|------|----------|--------------|------------|---------|
| 企画 | 何を作るか決める | 企画書 | Consultant | ユーザー承認 |
| 要件定義 | 何が必要か整理 | 要件定義書 | Consultant | ユーザー承認 |
| 設計 | どう作るか決める | 設計書、ADR | Architect | ユーザー承認 |
| 実装 | 作る | コード、単体テスト | Coder | テストパス |
| テスト | 品質確認 | テスト結果 | QA | 品質基準クリア |
| 運用準備 | 本番準備 | インフラ、Runbook | SRE | デプロイ完了 |

【フェーズ移行】

- 完了条件を満たしたら、PMがユーザーに確認
- ユーザー承認後、次フェーズに移行
- project-state.json の phase を更新

フェーズは厳密な順序ではない！

- 必要に応じて前のフェーズに戻る
- 並行して進めることもある
- AIが状況を見て判断する